

Putting noexcept(auto) on hold, again

Abstract

EWG liked the idea of having noexcept(auto) in Lenexa. Bjarne voiced concern about making brittle interfaces easier to author. Agustin Berge pointed out that the apparent savings in explicit specification don't work when SFINAE on return type is desirable.

Rumination

Let's repeat the usual example of where a noexcept-specification repeats the expression in the function body:

```
template <class T, class U>
auto myplus(T&& t, U&& u) noexcept(noexcept(declval<T>()+declval<U>())) -> decltype(t+u)
{
    return t+u;
}
```

The main argument for noexcept(auto) is avoiding repetitions of the expression in the body, and an additional argument is that since return type deduction allows avoiding repeating the expression in a trailing return type, it would be nice to avoid repeating it in the noexcept-specifier.

The problem is that return type deduction will not help us. The practical examples of code like that in transparent operator-like functors and swap functions want to SFINAE on the return type, so we can't get rid of that repetition.

To summarize, noexcept(auto) would probably go some ways towards making transparent operator-like functors and swap functions prettier, but that serves an allegedly small crowd of programmers, while making writing brittle interfaces much easier for non-experts. Overall, that doesn't seem like the best of motivations for a language extension. I am therefore not going to propose going further with noexcept(auto) at this point.